

Part A - Theory question.

1. Python is a high-level, interpreted, general-purpose programming language.

Features:-

- easy to learn
- interpreted
- dynamically typed
- object-oriented, portable and
- open-source.

2. Compiled: The whole program is translated before execution. example: C/C++.

Interpreted: Code is executed by an interpreter.
Python is commonly called interpreted because its code is executed through the python runtime.

3. Identifier: Name of a variable, function or class.
Example: name.

Keyword: Reserved Python word.

Example: if, else, for, while, class, def.

4. (i). int - stores whole numbers.

(ii). float - stores decimal numbers.

(iii). complex - stores complex numbers

(iv). list - ordered and ~~un~~changeable collection.

(v). tuple - ordered and unchangeable collection.

5. Mutable: Can be changed after creation.
Examples: list, set, dictionary.

Immutable: Cannot be changed.

Examples: int, float, string, tuple.

6. Arithmetic: +, -, *, /, %, //, **

Relational: =, !=, >, <, >=, <=

Logical: and, or, not

Assignment: =, +=, -=, *=, /= etc.

7. Type casting converts one data type into another.

Implicit: Python performs the conversion automatically.

Explicit: Programmer performs it using functions
Such as int(), float() and str().

8. if: checks one condition

if-else: provides two choices.

if-elif-else: checks multiple conditions.

9. for: used to iterate over a sequence or range

while: Repeats while a condition remains true

10. Break: Stops the loop.

Continue: Skips the current iteration.

Pass: Does nothing and acts as a placeholder.

11. A function is a reusable block of code - It is defined using def, can accept parameters and can return a value using return.

12. Local: Declared inside a function and normally available only there.

Global: Declared outside functions and can be accessed throughout the program.

13. Recursion is when a function calls itself.

Advantage: Simplifies some problems.

Disadvantage: Uses additional call-stack memory and may reach recursion limits.

14. List: Ordered and mutable.

Tuple: Ordered and unchangeable collection, immutable.

Set: Stores unique elements.

Dictionary: Stores key-value pairs.

15. List Slicing extracts part of a list using

`list[start:stop:step]`

Example: `a = [10, 20, 30, 40]; a[1:3]` gives `[20, 30]`.

16. Difference are:-

`append()`: adds one item at the end.

`extend()`: adds multiple items.

`insert()`: adds an item at a specified position.

17. A dictionary is a mutable collection of key-value pairs.

Example: `{'name': 'Gokul', 'mark': 90}`. Keys are unique.

18. For sets A and B:

Union: $A \cup B$; Intersection: $A \cap B$; Difference: $A - B$

Symmetric difference: $A \oplus B$.

19. Exception handling manages runtime errors.
try contains risky code.
except handles error,
else runs when no error occurs
finally always runs.

20. class: Blueprint for objects.

object: Instance of a class.

Constructor: `__init__()` initializes an object.

Inheritance: Child class gets features from parent.

Polymorphism: Same method/interface can have different implementations.

PART B - Basic programs.

21. `a = float(input())`

`b = float(input())`

`Print("Addition =", a+b)`

`Print("Subtraction =", a-b)`

`Print("Multiplication =", a*b)`

`if b != 0:`

`Print("Division =", a/b)`

`else:`

`Print("cannot divide by zero").`

22. `n = int(input())`

`if n%2 == 0:`

`Print("Even")`

`else:`

`Print("odd").`

23. `a, b, c = map(int, input().split())`

`if a >= b and a >= c:`

`print(a)`

`elif b >= a and b >= c:`

`print(b)`

`else:`

`print(c)`

24. `year = int(input())`

`if year % 400 == 0 or (year % 4 == 0 and year % 100 != 0):`

`print("Leap year")`

`else:`

`print("Not a Leap year").`

25. `p = float(input())`

`r = float(input())`

`t = float(input())`

`SI = p * r * t / 100`

`print("Simple Interest =", si).`

Loop and Number Programs.

26. `n = int(input())`

`total = 0`

`for i in range(1, n+1):`

`print(i, end=" ")`

`total += i`

`print()`

`print("sum = ", total).`

27. `n = int(input())`

`for i in range(1, 11):`

`Print(n, "x", i, "=", n*i)`

28. `n = int(input())`

`fact = 1`

`for i in range(1, n+1):`

`fact *= i`

`Print("Factorial = ", fact)`

29. `n = int(input())`

`Prime = n >= 2`

`for i in range(2, int(n**0.5)+1):`

`if n % i == 0:`

`Prime = False`

`break.`

`Print("Prime" if Prime else "Not prime")`

30. `n = int(input())`

`a, b = 0, 1`

`for i in range(n):`

`Print(a, end=" ")`

`a, b = b, a+b`

31. `n = int(input())`

`temp = n`

`digits = len(str(n))`

`total = 0`

`while temp > 0:`

`d = temp % 10`

`total += d ** digits`

`temp //= 10`

`Print("Armstrong Number" if total == n else "Not Armstrong")`

32. `n = int(input())`

`original = n`

`reverse = 0`

`while n > 0:`

`reverse = reverse * 10 + n % 10`

`n //= 10`

`Print ("palindrome" if original == reverse else
"Not a palindrome").`

33. `n = abs(int(input()))`

`total = 0`

`while n > 0:`

`total += n % 10`

`n //= 10`

`Print ("Sum of digits = ", total)`

34. `n = int(input())`

`reverse = 0`

`while n != 0:`

`reverse = reverse * 10 + n % 10.`

`n //= 10`

`Print (reverse).`

35. `a, b = map(int, input().split())`

`for n in range(a, b+1):`

`if n < 2: continue`

`Prime = True`

`for i in range(2, int(n**0.5) + 1):`

`if n % i == 0:`

`Prime = False`

`break`

`if Prime: Print (n, end=" ").`

PART C - String Programs

36. `S = input()`

`rev = ""`

`for ch in S:`

`rev = ch + rev`

`Print (rev)`

37. `S = input()`

`rev = S[::-1]`

`Print ("Palindrome " if S == rev else "Not a palindrome")`

38. `S = input()`

`v = c = d = sp = 0`

`for ch in S:`

`if ch.lower() in "aeiou": v += 1`

`elif ch.isalpha(): c += 1`

`elif ch.isdigit(): d += 1`

`elif ch.isspace(): sp += 1`

`Print ("Vowels = ", v).`

`Print ("consonants = ", c)`

`Print ("Digits = ", d)`

`Print ("space = ", sp)`

39. `S = input()`

`freq = {}`

`for ch in S:`

`freq[ch] = freq.get(ch, 0) + 1`

`for ch, count in freq.items():`

`Print (ch, ":", count).`

From:

```
40. S1 = input().replace(" ", "").lower()
    S2 = input().replace(" ", "").lower()
    Print ("Anagrams" if sorted(S1) == sorted(S2)
           else "Not Anagrams")
```

PART D - List, Set, and Dictionary programs.

```
41. a = list(map(int, input().split()))
    Print ("Largest =", max(a))
    Print ("Smallest =", min(a))

42. a = list(map(int, input().split()))
    U = list(set(a))
    if len(U) < 2:
        Print ("second largest does not exist")
    else:
        U.sort()
        Print ("second largest =", U[-2])

43. a = list(map(int, input().split()))
    result = []
    for x in a:
        if x not in result:
            result.append(x)
    Print (result)

44. a = list(map(int, input().split()))
    b = list(map(int, input().split()))
    common = []
    for x in a:
        if x in b and x not in common:
            common.append(x)
    Print (common)
```

```

45. A = Set (map (int, input().split()))
    B = Set (map (int, input().split()))
    Print ("Union = ", A|B)
    Print ("Intersection = ", A & B)
    Print ("Difference = ", A-B)
    Print ("Symmetric difference = ", A^B)

```

```

46. n = int (input())
    Students = {}
    for i in range (n):
        name = input()
        mark = float (input())
        Students [name] = mark
    name = max (Students , key = students.get)
    Print ("Highest: ", name, Students [name])

```

PART E - Functions, oop and File Handling.

```

47. def is_prime (n):
    if n < 2: return False
    for i in range (2, int (n**0.5) + 1):
        if n % i == 0: return False
    return True
n = int (input())
Print ("Prime" if is_prime (n) else "Not prime")

```

```

48. def factorial(n):
    if n <= 1: return 1
    return n * factorial(n-1)
n = int(input())
print(factorial(n))

```

```

49. class student:
    def __init__(self, name, reg, marks):
        self.name = name
        self.marks = marks
    def total(self): return sum(self.marks)
    def average(self): return self.total() / len(self.marks)

```

```

name = input()
reg = input()
marks = list(map(float, input().split()))
s = student(name, reg, marks)
print("Name =", s.name)
print("Register Number =", s.reg)
print("Total =", s.total())
print("Average =", s.average())

```

```

50. filename = input()
try:
    with open(filename, "r") as f:
        text = f.read()
        print("Lines =", len(text.splitlines()))
        print("Words =", len(text.split()))
        print("Characters =", len(text))
except FileNotFoundError:
    print("File not found")

```